```
//namespace Program
//{
//    internal class Program
//    {
//        static void Main(string[] args)
//        {
//            Console.WriteLine("Hello, World!");
//        }
//    }
//}
//using System;

//namespace Program
//{
//    class program1
//    {
//        public static void Main(String[] args)
//        {
//            int cp, sp, p, l;
//            Console.WriteLine("Enter Cost Price");
//            cp = Convert.ToInt32(Console.ReadLine());

//            Console.WriteLine("Enter  Selling Price");
//            sp = Convert.ToInt32(Console.ReadLine());

//            if (sp > cp)
//            {
//                p = sp - cp;
//                Console.WriteLine("Gain amount " + p);
//            }
//            else
//            {
//                l = cp - sp;
//                Console.WriteLine("Loss amount " + l);
//            }
//            Console.ReadKey();
//        }
//    }
```

```
//}


//WAP for finding greatest numhber among 4 numbers

//using System;
//namespace Program
//{
//    class biggest
//    {
//        public static void Main(string[] args)
//        {
//            int a, b, c, d, temp;
//            Console.WriteLine("Enter the first Number");
//            a = Convert.ToInt32(Console.ReadLine());

//            Console.WriteLine("Enter the second Number");
//            b = Convert.ToInt32(Console.ReadLine());

//            Console.WriteLine("Enter the third Number");
//            c = Convert.ToInt32(Console.ReadLine());

//            Console.WriteLine("Enter the fourth Number");
//            d = Convert.ToInt32(Console.ReadLine());

//            if (a > b && a > c && a > d)
//            {
//                Console.WriteLine(a + " is greatest Number");
//            }
//            else if (b > a && b > c && b > d)
//            {
//                Console.WriteLine(b + " is greatest Number");
//            }
//            else if (c > a && c > b && c > d)
//            {
//                Console.WriteLine(c + " is greatest Number");
//            }
//            else
```

```
//            {
//                Console.WriteLine(d + " is greatest Number");
//            }
//        }
//    }
//}




//WAP for finding the sum of array

//using System;
//namespace program
//{
//    class sumOfArray
//    {
//        public static void Main(String[] args)
//        {
//            int i, n, sum1 = 0, sum2 = 0;
//            int[] number = new int[100];
//            Console.WriteLine("Enter the size of the array");
//            n = Convert.ToInt32(Console.ReadLine());

//            for (i = 0; i < n; i++)
//            {
//                Console.WriteLine("Enter the value of " + (i + 1) +" element");
//                number[i] = Convert.ToInt32(Console.ReadLine());

//            }

//            //for sum long method
//            for (i = 0; i < n; i++)
//                sum1 += number[i];
//            Console.WriteLine("The sum of the array is " + sum1);

//            // short and inbulid method

//            sum2 = number.Sum();
```

```
//            Console.WriteLine("The sum of the arrray is " + sum2);
//        }
//    }
//}



//WAP to find the largest,smallest and the average of the array

//using System;
//namespace program
//{
//    class findAll
//    {
//        public static void Main(String[] args)
//        {
//            int i, n, average, smallest, biggest, sum = 0;
//            int[] number = new int[100];
//            Console.WriteLine("Enter the size of the array");
//            n = Convert.ToInt32(Console.ReadLine());

//            for (i = 0; i < n; i++)
//            {
//                Console.WriteLine("Enter the " + (i + 1) + " element");
//                    number[i] = Convert.ToInt32(Console.ReadLine());
//            }
//            biggest = number[0]; smallest = number[0];

//            for (i = 0; i < n; i++)
//            {
//                if (number[i] > biggest)
//                    biggest = number[i];
//                if (number[i] < smallest)
//                    smallest = number[i];
//                sum += number[i];
//            }
//            average = sum / n;

//            //print
```

```
//            Console.WriteLine("The biggest number of the array is " + biggest);
//            Console.WriteLine("The smalllest number of the array is " + smallest);
//            Console.WriteLine("The sum number of the array is " + sum);
//            Console.WriteLine("The average number of the array is " + average);

//            // print the bigesst,smallest,sum using the inbulit method
//            Console.WriteLine("The biggest number of the array is " + (number.Max()));
//            Console.WriteLine("The smallest number of the array is " + (number.Min()));
//            Console.WriteLine("The sum number of the array is " + (number.Sum()));
//        }
//    }
//}



//WAP to sort an array using both inbuilt method and  logic method

//using System;
//namespace program
//{
//    class sortArrray
//    {
//        public static void Main(String[] args)
//        {
//            int i, n, j, temp;
//            int[] numbers = new int[100];
//            Console.WriteLine("Enter the size of array");
//            n = Convert.ToInt32(Console.ReadLine());

//            for (i = 0; i < n; i++)
//            {
//                Console.WriteLine("Enter the " + (i + 1) + "th element");
//                numbers[i] = Convert.ToInt32(Console.ReadLine());

//            }

//            for (i = 0; i < n; i++)
//            {
//                for (j = i+1; j < n; j++)
```

```
//             {
//                 if (numbers[j] < numbers[i])
//                 {
//                     temp = numbers[i];
//                     numbers[i] = numbers[j];
//                     numbers[j] = temp;
//                 }
//             }
//         }

//         //for print
//         Console.WriteLine("Array after sorting");
//         for (i = 0; i < n; i++)
//         {
//             Console.WriteLine(numbers[i]);
//         }

//         // using bulit in with foreach the builtin method is only used when the array is already give not a user input array
//         //Array.Sort(numbers);
//         //foreach(int a in numbers){
//         //    Console.WriteLine(a);
//         //}
//     }
// }
//}



// program for item search in an array
//using System;
//namespace program
//{
//   class itemSearch
//   {
//       public static void Main(String[] args)
//       {
//           int i, n, item;
//           bool flag=false;
//           int[] numbers = { 1, 2, 3, 4, 5 };
```

```
//            n = numbers.Length;
//            Console.WriteLine("Enter the search element");
//            item = Convert.ToInt32(Console.ReadLine());
//            for (i = 0; i < n; i++)
//            {
//                if (item == numbers[i])
//                {
//                    Console.WriteLine("Elements found in " + i + "th index");
//                    flag = true;
//                    return;
//                }
//  }
//            if (!flag)
//            {
//                Console.WriteLine("No element found in the array");
//            }
//}
//}
//}



//2d array rectangle
/*
using System;

namespace Program
{
   class _2darray
   {
      public static void Main(String[] args)
      {
         int i, j;
         int[,] numbers = new int[2, 2];
         Console.WriteLine("Enter the elements in 2 d array ");
         for (i = 0; i < numbers.GetLength(0); i++)
         {
            for (j = 0; j < numbers.GetLength(1); j++)
            {
```

```
                numbers[i, j] = Convert.ToInt32(Console.ReadLine());
            }
        }

        //for read
        Console.WriteLine("The elements of the array");
        foreach(int a in numbers)
        {
            Console.WriteLine(a);
        }
      Console.ReadKey();
      }
   }

}

*/

// class and object

//using System;

//namespace personalDetails
//{
//    class program
//    {
//        public static void Main(String[] args)
//        {
//            person p = new person();
//            p.id = 0001;
//                p.name="Birkam Gyawali";
//            p.age = 19;
//            p.address = "Kathmandu";
//                p.displayData();
//        }
//    }
```

```
//// create a class for the person


//    class person
//    {
//      public  int id, age;
//        public string name, address;


//          public  void displayData()
//        {
//            Console.WriteLine("Hello " + name + " Your id is " + id + " and you are " + age + " year old. " + " You are from  " + address);
//        }
//    }


//}



// class and object for the area ,volume and parameter of rectangle with multiple class
// now rewrite with single class



/* using System;
namespace AVPofRectangle
{
   class program
   {
      public static void Main(String[] args)
      {
         int l, b, h;
         Console.WriteLine("Enter the length of the rectangle");
         l = Convert.ToInt32(Console.ReadLine());
         Console.WriteLine("Enter the breadth of the rectangle");
         b = Convert.ToInt32(Console.ReadLine());
         Console.WriteLine("Enter the height of the rectangle");
         h = Convert.ToInt32(Console.ReadLine());


         area A = new area();
         A.Calculate(l, b);
```

```
            volume v = new volume();
            v.CaulculateVolume(l, b, h);
            Console.ReadKey();
        }
    }

    // class for area
    class area
    {

        public void Calculate(int l, int b)
        {
            int area = l * b;
            Console.WriteLine("The area of the rectangle is " + area);
        }

    }



    class volume
    {

        public void CaulculateVolume(int l, int b, int h)
        {
            int volume = l * b * h;
            Console.WriteLine("The volume of rectangle is " + volume);
        }
    }
}
*/

//using single class

/*using System;
namespace AVPSingleClass
{
```

```
class program
{
    public static void Main(String[] args)
    {
        Caculation c = new Caculation();
        Console.WriteLine("Enter the length of the rectangle");
        double l = Convert.ToDouble(Console.ReadLine());
        Console.WriteLine("Enter the breadth of the rectangle");
        double b = Convert.ToDouble(Console.ReadLine());
        Console.WriteLine("Enter the height of the rectangle");
        double h = Convert.ToDouble(Console.ReadLine());
        c.setData(l, b, h);
        Console.WriteLine("The area of the rectangle is " + c.area());
        Console.WriteLine("The volume of the rectangle is " + c.volume());

        Console.ReadKey();
    }
}

class Caculation
{
    double l, b, h;
    public void setData(double l, double b, double h)
    {
        this.l = l;
        this.b = b;
        this.h = h;
    }

    public double area()
    {
        return l * b;
    }

    public double volume()
    {
        return l * b * h;
    }
```

```
    }

}
*/

/*ACCESS MODIFIER
Note :
    1. class is by default internal and never by private
    2. members inside class is by default private
    3. method is also by default private
*/

/*

// claculation of simple interes

using System;
namespace SimpleInterest
{
    class Calculatin
    {
        public double SICalculation(double p, double t, double r)
        {
            return (p * t * r) / 100;


        }
    }
    class program
    {
        public static void Main(string[] args)
        {
            Calculatin c = new Calculatin();
            Console.WriteLine("The simple interest is " + c.SICalculation(20000, 3, 4));
        Console.ReadKey();
        }
    }
}
*/
```

/*

example of call by vlaue : this is the default way of call method

definition : When you pass a variable to a method,

C# makes a copy of that value. The method works on the copy. The original variable is completely safe and never touched.

```
using System;
class program
{
   static void Show(int x)
   {
      x = x + 10;
      Console.WriteLine("Inside the value " + x);
   }
   public static void Main(String[] args)
   {
      int a = 20;
      Show(a);
      Console.WriteLine("Outside method value" + a);
   }
}
```

Call by reference : Instead of sending a copy, you send the actual memory address of the variable. Whatever the method does to it directly changes your

original variable.

```
using System;
class program
{
   static void show (ref int x )
   {
      x = x + 10;
      Console.WriteLine("Inside the function value {0}", x);
   }

  public  static void Main(String[] args)
   {
      int a = 10;
```

```
        show(ref a);
        Console.WriteLine("Outside the function vlaue {0}", a);
        Console.ReadKey();
    }
}
```

Output parmater :

Definition : This is like ref but done specially for output.the method fills the variable for you . we dont need to intialize before call in fact the method is required to assign it before returning.

where to use it : needs to produce multiple results

we assign the vlaue inside the method in out but not in the ref and the value;

and dont assign any value inside the main class in out but iin the ref and the value we assign the value

```
using System;
 internal class program
{
    static void Show( out int x)
    {
        x = 50; // assign value inside the method
        Console.WriteLine("Inside the method "+x);
    }

    static void Main(String[] args)
    {
        int a; // we dont need to assign vlaue fo the a
        Show(out a);
        Console.WriteLine("Outside the function " + a);
        Console.ReadKey();
    }
}
```

calss by params

definitionn: mthod accept any number of argument of the same types.

Why it exists: You often don't know at design time how many values will be passed. params removes the need to create an array manually before every call.

we can pass any number of the parameter we dont need to assign the array

```
using System;

internal class program
{
    static void show(params int[] num)
    {
        int total = 0;
        foreach(int n in num)
            total += n;
        Console.WriteLine("Total : " + total);
    }

    static void Main(String[] args)
    {
        show(1, 2);
        show(1, 2, 3);
        Console.ReadKey();
    }
}
```

Method overloading

```
using System;
namespace simpleMethodOverloading
{
    internal class program
    {
        public void add(int a, int b)
        {
            int c = a + b;
            Console.WriteLine("The sum of 2 numbers is {0} ", c);

        }
        public void add(int x, int y, int z)
```

```
        {
            int sum = x + y + z;
            Console.WriteLine("The sum of three numbers is {0} ", sum);
        }
        public static void Main(string[] args)
        {
            program p = new program();
            p.add(2, 3);
            p.add(2, 3, 4);
            Console.ReadKey();
        }
    }
}
```

with return of int

```
namespace simpleMethodOverloading
{
internal class program
    {
        public int add(int a, int b)
        {
            //int c = a+b;
            //Console.WriteLine("The sum of 2 numbers is ", c);
            return a + b;

        }
        public int add(int x,int y, int z)
        {
            //int sum = x + y + z;
            //Console.WriteLine("The sum of three numbers is ", sum);
            return x + y + z;
        }
        public static void Main(string[] args)
        {
            program p = new program();
            //p.add(2, 3);
            Console.WriteLine(p.add(2, 3));
```

```
            Console.WriteLine(p.add(2, 3, 4));
            //p.add(2, 3, 4);
            Console.ReadKey();
        }
    }
}
*/

/*simple inheritance

using System;

namespace simpleInheritance
{
  internal class program
    {
        class A
        {
            public void display()
            {
                Console.WriteLine("hello user");
            }
        }

        class B : A
        {
            public void test()
            {
                Console.WriteLine("this is test");
            }
        }

       class Inheritance
        {
            public static void Main(String[] args)
            {
                B b = new  B();
                b.display();
```

```
            b.test();
            Console.ReadKey();
        }
    }

}
}

*/


/* write a program to base class name employe has salary 35000 and who is manager get bonus 20000 per month. uisng inheritance


using System;
namespace example
{
    internal class program
    {
        class employer
        {
         public  int salary = 30000;
            public void Salary()
            {
                Console.WriteLine("Salay is  {0}", salary);
            }
        }
        class manager : employer
        {
          public int bonous = 20000;
          public int total ;


            public manager()
            {
                total = salary + bonous;
            }
            public void managerSalary()
            {
```

```
            Console.WriteLine("Manager salary is {0}",total);
        }

    }

    class calulation
    {
        public static void Main(String[] args)
        {
            manager m = new manager();
            m.Salary();
            m.managerSalary();
            Console.ReadKey();
        }
    }

}
```

consustructor

i. Default Constructor

```
using System;
namespace defaultConstructor
{

    public class Multiplication
    {
        public int a, b;
        public Multiplication(){
            a=20;
            b=30;
        }
    }
```

```
    public class program
    {
        public static void Main(String[] agrs)
        {
            Multiplication m = new Multiplication();
            Console.WriteLine(m.a);
            Console.WriteLine(m.b);
        }
    }

}
```

ii. Instance/Paramterize Constructor

```
using System.Xml.Serialization;

public class program
{
    int x, y;
    program(int a,int b)
    {
        x = a;
        y = b;
    }

    void add()
    {
        int c= x+y;
        Console.WriteLine(c);
    }

    public static void Main(String[] args)
    {
        program p = new program(2, 3);
        p.add();
    }
}
```

iii. Overload constructor

```
class program
{
    int mul;
    program(int a,int b)
    {
        mul = a * b;
        Console.WriteLine("The mul of 2 number is {0}", mul);
    }

    program(int a,int b, int c)
    {
        mul = a * b*c;
        Console.WriteLine("The multiplication of 3 numbers is" + mul);
    }

    public static void Main(String[] args)
    {
       new program(2, 3);
        new program(2, 3, 4);
        Console.ReadKey();
    }
}
```

iv. Static constuctor

```
class program
{
    public program()
    {
        Console.WriteLine("Run when we call it ");
    }
    static program()
    {
```

```
        Console.WriteLine("Run for one time when the instance of class is created \n Dont accept any access modifier");


    }

    public static void Main(String[] args)
    {
        program p= new program(); // both the constructor are called


        new program(); // escape static as it run for only one time



    }
}
```

Example of the destructor

```
using System;
internal class program
{
    public program(String message)
    {
        Console.WriteLine(message);
    }
    ~program()
    {
        Console.WriteLine("call destuctor");
    }
    public void afterDestructor()
    {
        Console.WriteLine("This method is called after destructor");
    }




    public static void Main(String[] args)
    {
```

```
        string message = "Call from constructor";
        program p = new program(message);
        p.afterDestructor();


        Console.ReadKey();
    }
}
```

using this keyword to call another constructor

```
using System;

public class detail
{
    int id;
    string name;
    public detail(int id,string name)
    {
        this.id = id;
        this.name = name;
        Console.WriteLine("Call first constuctor");
    }

    public detail(int id):this(id,"Bikram")
    {
        Console.WriteLine("Name set to default");
        Console.WriteLine("Call first constuctor");
    }
    public void show()
    {
        Console.WriteLine("Your id is {0} and name is {1}", id, name);
    }
}

public class program
```

```
{
   public static void Main(String[] args)
   {
      detail d= new detail(2);
       d.show();
       Console.ReadKey();
   }
}
```

==========example of get and set==========

```
internal class program
{
   private int number;
   public int score
   {
      get
      {
         return number;
      }
      set
      {
         number = value;
      }
   }
   public void display()
   {
      Console.WriteLine("The number is {0}", number);
   }
}
   internal class test
   {
```

```
    public static void Main(String[] args)
    {
        program p = new program();
        p.score = 20;  // we assign the value to the private field of another class
        //p.display();
        Console.WriteLine("The number is {0}", p.score);
    }
}
```

automatic properites

==========display your information using the properties

```
public class detail {
    private string name;
        public string sName
    {
        get
        {
            return name;
        }
        set
        {
            name= value;
        }
    }

    public int age { get; set; } // this is the automatic method

    public string address {  get; set; }   // the address is string

    public void display()
```

```
    {
        Console.WriteLine("Your nam is {0} and your age is {1}. You are from {2}", name, age, address);
    }
}



internal class program
{
    public static void Main(String[] args)
    {
        detail d = new detail();

        Console.WriteLine("Enter your name , age  and addresss one by one");
        d.sName = Console.ReadLine();
        d.age=Convert.ToInt32(Console.ReadLine()); // or we can write d.age=int.Parse(Console.ReadLine());
        d.address = Console.ReadLine();
        d.display();
        Console.ReadKey();

    }
}



=======   display your family details ==
namespace PropertiesExample
{
    class Student
    {
        private int _StdId;
        private string _Name;
        private string _Fname;

        public int StdId
        {
            set
            {
                if (value <= 0)
```

```
        {
            Console.WriteLine("The ID can not be zero or negative");

        }
        else
        {
            _StdId = value;



        }

    }
    get
    {
        return _StdId;

    }

}
public string Name
{
    set
    {
        if (string.IsNullOrEmpty(value))
        {
            Console.WriteLine("Please Enter Your Name");
        }
        else
        {
            _Name = value;
        }


    }
    get
    {
        return _Name;
```

```
        }
    }
    public string Fname
    {
        set
        {
            if (string.IsNullOrEmpty(value))
            {
                Console.WriteLine("Plz Enter your father name");
            }
            else
            {
                _Fname = value;
            }

        }
        get
        {
            return _Fname;
        }
    }
}
internal class Program
{
    static void Main(string[] args)
    {
        Student s = new Student();
        s.StdId = 5;
        s.Name = "Mahesh";



        s.Fname = "Binod";
        Console.WriteLine("My ID=" + s.StdId);
        Console.WriteLine("My  name is=" + s.Name);
        Console.WriteLine("My father name is=" + s.Fname);
        Console.ReadKey();
```

```
        }
    }
}
```

==========indexier example==========

```
public class Student
{
    private int[] marks = new int[10];

    public int this[int i]{
        get { return marks[i]; }
        set { marks[i] = value; }
    }
}
class program
{
    public static void Main(String[] args)
    {
        Student s = new Student();
        s[0] = 40;
        s[1] = 70;
        Console.WriteLine(s[0]);
        Console.WriteLine(s[1]);
        Console.ReadKey();
        // this method is for the loop
        //for (int i = 0; i < 10; i++)
        //  {
        //      s[i] = i * 3;
        //  }

        //  for(int i=0;i<10;i++){
        //      Console.WriteLine(s[i]);
        //  }
```

```
    }
}
```

========== static classes ==========

```
static class detail
{
   public  static string name;
  public  static int age;

    public static void show()
    {
        Console.WriteLine("Your name is {0} and your age is {1}", name, age);
    }
}

class program
{
    public static void Main(String[] agrs)
    {
        detail.name = "Bikram Gyawali";   // we dont need to creat the object  fo the staic class
        detail.age = 20;   // the member of the static clsss must ne static to be access by other class
        detail.show();
        Console.ReadKey();
    }
}
```

========== finalizer ==========

```
class fileFinalizer
{
    public fileFinalizer()
```

```
    {
        Console.WriteLine("File is opened");
    }


    public void readFile()
    {
        Console.WriteLine("File read");
    }

    ~fileFinalizer()
    {
        Console.WriteLine("Call finalizer");
    }
}

class program
{
    public static void Main(String[] args)
    {
        fileFinalizer f = new fileFinalizer();
        f.readFile();
        f = null;
        GC.Collect();
        Console.ReadKey();
    }
}
```

========== struct or structure ==========

this doesnt suppport inheritance

```
using System.Xml.Linq;

struct student
{
    public int age;
    public string name;
```

```
}

class program
{
  public  static void Main(String[] args)
   {
      //student s = new student();   we can use this also
      student s;
      s.age=20;
      s.name="Bikram Gyawali";
      Console.WriteLine("Name = {0} and age = {1}", s.age, s.name);
      Console.ReadKey();
   }
}
```

========== base keyword=======

base is a keyword that refers to the parent class (base class) from inside a child class.

It lets you access the parent's members that are hidden or overridden by the child. 3 use of base class

1. call parent class method
2. call parent class constructor
3. call parent class method with overide

```
public class person
{
   public int age;
   public string name;

   public person(int age,string name)
   {
      this.age = age;
      this.name = name;
```

```
        Console.WriteLine("Call student constructor");
    }

    public virtual void show() // virtual keyword allow for override
    {
        Console.WriteLine("Name = {0} \n and age = {1}", name, age);
    }
}

public class student : person
{
    public int id;
    public student(int id,int age,string name) : base(age, name)//this call the parent constructor and set value of age and name
    {
        this.id = id;
            Console.WriteLine("Call person constructor");
     }

    // override the method

    public override void show() // override keyword is necessary
    {
        base.show(); // call the parent class method without override
        Console.WriteLine("data of student \n");
        Console.WriteLine("id = {0} \n name={1} \n age={2}",id,name, age);
    }

    public void showName()
    {
        Console.WriteLine("Name = {0} ", base.name); //access the parent class field
    }
}

class program
{
    public static void Main(String[] args)
    {
        student s = new student(2, 20,"Bikram Gyawali");
```

```
        Console.Write("display data \n");
        s.show();
        Console.Write("display base class name field");
        s.showName();
        Console.ReadKey();    }
}
```

========== inheritance ==========

==========1. Multilevel inheritance ==========

```
public class Number
{
    public int a, b, sum, diff;
    public void  ReadNumber(int x,int y)
    {
        a = x; b = y;
        Console.WriteLine("a={0}\n b={1}", a, b);
    }
}

public class Add:Number
{
    public void add()
    {
        sum = a + b;
        Console.WriteLine("Addition"+sum);
    }
}

class Sub: Add
{
    public void sub()
    {
```

```
        diff = a - b;
        Console.WriteLine("Subtraction"+diff);
    }
}

class program
{
    public static void Main(String[] args)
    {
      Sub s = new Sub();
        s.ReadNumber(4, 10);
        s.add();
        s.sub();
        Console.ReadKey();
    }
}
```

========== Hierarchical inheritance ==========

```
class Calculation {
  public int a, b;
    public void ReadDimension(int a, int b)
    {
        this.a=a;
        this.b = b ;
    }
}

 class Rectangle : Calculation
{
    public void AreaRect()
    {
        base.ReadDimension(10, 5);
        int area = a * b;
        Console.WriteLine("Area" + area);
    }
```

```
}

class Triangle : Calculation
{
   public void AreaTri()
   {
      base.ReadDimension(10, 5);
      double area = 0.5 * a * b;
      Console.WriteLine("Area of triangle" + area);
   }
}

class program
{
   public static void Main(String[] args)
   {
      Triangle t = new Triangle();
      t.AreaTri();
      Rectangle r = new Rectangle();
      r.AreaRect();
      Console.ReadKey();
   }
}



        ======= multiple inheritance ===
using System;

interface Bird
{
    void eat();
}
interface  Dog
{
    void Bark();
}
class Animal : Bird, Dog
{
```

```
    public void eat()
    {
        Console.WriteLine("Eat food");
    }
    public void Bark() {
        Console.WriteLine("Bark");
    }
}
class Program
{
    public static void Main(String[] args)
    {
        Animal a = new Animal();
        a.eat();
        a.Bark();

        Console.ReadKey();
    }
}
```

======== abstract class and method =====

we cannt create a object for the abstract class because it may include the abstract method without implementation

```
abstract class Shape
{
    public abstract void area();
    public void display()
    {
        Console.Write("This is method without abstraction \n");
    }
}

class Circle : Shape
```

```
{
    int r;
    public Circle(int r)
    {
        this.r = r;
    }


    public override void area()
    {
        double a = 3.14 * r * r;
        Console.WriteLine("The area of the circle is " + a);
    }
}

 class program
{
    public static void Main(String[] args)
    {
        Circle c = new Circle(2);
        c.display();
        c.area();
        Console.ReadKey();
    }
}
```

1. Write a program to implement abstraction using an abstract class

BankAccount with appropriate abstract properties and methods.

Create suitable derived classes to calculate interest for different account types and display the results.

```
public abstract class BankAccount{
    public abstract string AccountType { get; }

    public abstract double CalculateInterest(double amount);
```

```
    public void Display()
    {
        Console.WriteLine("This is bank account");
    }
}

class SavingAccount : BankAccount
{
    public override string AccountType
    {
        get
        {
            return "Saving Account";
        }
    }

    public override double CalculateInterest(double amount)
    {
        return amount * 0.5;
    }

}

class CurrentAccount : BankAccount
{
    public override string AccountType{ get
        {

            return "Current Account";
        }
    }

    public override double CalculateInterest(double amount)
    {
        return 0;
    }
}
```

```
class program
{
   public static void Main(String[] args)
   {
      BankAccount ba = new SavingAccount();  // we cannt create the class of abstract class
                                             // so we create object of inheritate class . this is also called upcasting
      Console.WriteLine("Account Type " + ba.AccountType);
      Console.WriteLine("Interest amount " + ba.CalculateInterest(1000));




      BankAccount bc = new CurrentAccount(); // same reason of above
      Console.WriteLine("Account Type "+ bc.AccountType);
      Console.WriteLine("Interest Amount  " + bc.CalculateInterest(110));


      Console.ReadKey();



   }
}
```

========== INTERFACE ==========

1. Write a C# program to implement an interface named IPayment that is used by EsewaPayment and KhaltiPayment classes.

```
interface  IPayment
{
    void Payment(double amount);
}

class Esewa : IPayment
{
  public void Payment(double amount)   // its compulsory to make the method public
   {
      Console.WriteLine("The amount paid by esewa is " + amount);
   }
```

```
}

class Khalti : IPayment
{
    public  void Payment(double amount)
    {
        Console.WriteLine("The amount paid by the khalti is " + amount);
    }
}

 class program
{
    public static void Main(String[] args)
    {
        Esewa ip = new Esewa();
        ip.Payment(900);

        IPayment pi = new Khalti();
        pi.Payment(300);

        Console.ReadKey();
    }
}
```

2. Calculate the area and permiter of the rectangle using the interface

```
 interface Calculation
{
    void getData(int l, int b);
    int Area();
    void Perimeter();
}

class Rectangle : Calculation
{
    int l, b;
```

```
    public void getData(int l,int b)
    {
        this.l = l;
        this.b = b;
    }

    public int Area()
    {
        return l * b;
    }
    public void Perimeter()
    {
        int p = 2 * (l + b);
        Console.WriteLine("The permiter is " + p);
    }
}

class program
{
    public static void Main(String[] args)
    {
        Rectangle re = new Rectangle();
        re.getData(4, 5);
        Console.WriteLine("The are of the rectangle is " + re.Area());
        re.Perimeter();
        Console.ReadKey();
    }
}
```

==========Polymorphism in C#

Compile Time

❖Method Overloading

❖Operator Overloading

Runtime

❖Method Overriding

❖Virtual Function

==========

1). Example program to demonstrate Online Shopping Delivery Charge .

```
class Delivery
{
    public void Charge()
    {
        Console.WriteLine("The delivery charge is 50 ");
    }

    public void Charge(int weight)
    {
        double charge = 50+ (weight*10);
        Console.WriteLine("Delivery charge with weight" + charge);
    }

    public void Charge(int weight, string location)
    {
        double charge = 50 + (weight * 10);
        if (location == "Kathmandu")  charge += 20;
        else charge += 30;

        Console.WriteLine("Delivery charge for the location {0} is rs {1}", location,charge);
    }
}

class program
{
    static void Main(String[] args)
    {
        Delivery d = new Delivery();
        d.Charge();
        d.Charge(20);
        d.Charge(10, "Kathmandu");
```

```
        d.Charge(2, "Butwal");
        Console.ReadKey();
    }
}
```

========== method overridding ==========

there must be virtual keyword in base class method and override keyword in the derive class method

1. WAP to implement a notification system using the methodoveride

```
class Notification
{
    public virtual void notificate()
    {
        Console.WriteLine("This is a notificaton sytem");
    }
}
class Email : Notification
{
    public override void notificate()
    {
        Console.WriteLine("This is email notification");
    }
}

class Sms : Notification
{
    public override void notificate()
    {
        Console.WriteLine("SMS notification");
    }
}

class program
{
    public static void Main(String[] args)
    {
```

```
        Notification n = new Notification();
        n.notificate();

        Email em = new Email();
        em.notificate();

        Sms s = new Sms();
        s.notificate();

        Console.ReadKey();
    }
}
```

========== VIRTUAL METHOD ==========

1. Wap to implement a virtual method

```
class VirtualMethod
{
    public virtual void message()
    {
        Console.WriteLine("The virtual method from base class");
    }
}

class dervideClass : VirtualMethod
{

    public override void message()
    {
        base.message();
        Console.WriteLine("This is a method of dervied class");
    }
}

class program
```

```
{
    public static void Main(String[] args)
    {
        dervideClass d = new dervideClass();
        d.message();
        Console.ReadKey();
    }
}
```

========Upcasting and downcasting ===========

wap to demostract the upcast and downcast

```
class Animal
{
    public void eat()
    {
        Console.WriteLine("Animal eats");
    }
}

class Dog : Animal
{
    public void bark()
    {
        Console.WriteLine("the dog bark");
    }
}

class Program
{
    public static void Main(String[] args)
    {
        //upcasting

        Animal a = new Dog();
        a.eat();
```

```
        //a.bark();  throw error as the method is hidden from the reference



        // another mehtod of upcasting



        //Dog d = new Dog();
        //Animal a = d;    // upcasting
        //a.eat();
        ////a.bark();   the animal is a reference and the method is inside the dog so it will throw error
        //d.bark(); // this work as we have the object of the dog as d refernce
        //downcast

        Dog dd =  (Dog)a;
        dd.eat();
        dd.bark();

        Console.ReadKey();
    }
}
```

========== Operator Overloading ==========

overloading unary operator

```
class Calculation
{
    public int a, b;
   public Calculation(int x, int y)
    {
        a = x;
        b = y;
    }

    public void display()
    {
```

```
        Console.WriteLine("a= {0} \n b={1}", a, b);
    }

    // operator overloading

    public static  Calculation operator- (Calculation c){
        //c.a = -c.a;
        //c.b=-c.b;
        //return c;   as this modify the real object
        return new Calculation(-c.a, -c.b);  // this will return new object
    }



}

class Program
{
    public static void Main(String[] args)
    {
        Calculation ca = new Calculation(20, 30);
        Console.WriteLine("Simple object data befor operator overload");
        ca.display();

        ca = -ca;
        Console.WriteLine("New object contains");
        ca.display();



        Console.ReadKey();
    }
}
```

========== ++ incrementar operator ==========

```
class counter
```

```
{
    public int x;
   public counter(int x)
    {
        this.x = x;
    }




    public static  counter operator ++(counter c)  // we can put any name in the object of the counter like (counter cccc)
    {
        return new counter(c.x++);
    }
    public void display()
    {
        Console.WriteLine("x = "+x);
    }
}

class Program
{
    public static void Main(String[] args)
    {
        counter c = new counter(40);
        Console.WriteLine("Value before operator overload");
        c.display();


        Console.WriteLine("Value before operator overload");


       c=c++; //using overload operator ++
        c.display();
        Console.ReadLine();
    }
}
```

==========Binary operator overloading ==========

wap to implement the +,-,* for compearing the number obtain by the operator

```
class Student
{
    public int marks;
    public string name;

    public Student(string name,int marks)
    {
        this.marks = marks;
        this.name = name;
    }

    // overload +

    public static Student operator +(Student s1,Student s2)
    {
        return new Student("Total/Combine", s1.marks + s2.marks);  // this become new object as Student(name,marks)
    }

    //Overload -
    public static Student operator -(Student s1,Student s2)
    {
        return new Student("Difference", s1.marks - s2.marks);
    }

    //overload *
    public static Student operator *(Student s1,int mul)
    {
        return new Student(s1.name, s1.marks * mul);
    }

    public void display()
    {
        Console.WriteLine("Name : {0}     marks : {1}", name, marks);
    }
}
```

```
class Program
{
    public static void Main(string[] args)
    {
        Student s1 = new Student("Bikram", 90);
        Student s2 = new Student("Hari", 85);

        // data of s1 student
        s1.display();

        //data of d2 student

        s2.display();

        // + operator call
        Student s4 = s1 + s2;
        s4.display();

        // - operator

        Student s5 = s1 - s2;
        s5.display();

        // * operator

        Student s6 = s1 * 2;
        s6.display();

        Console.ReadKey();
    }
}
```

2. Write a C# program to demonstrate operator overloading using + operator to add two account amount.

```
class Account
{
    public int amount;
```

```
    public Account(int amount)
    {
        this.amount = amount;
    }

    public static Account operator + (Account a1, Account a2){
        return new Account(a1.amount + a2.amount);
}

    public void display()
    {
        Console.WriteLine("Amount " + amount);
    }
}
class Program
{
    public static void Main(String[] args)
    {
        Account a1 = new Account(50000);
        a1.display();

        Account a2 = new Account(4000);

        a2.display();

        Console.WriteLine("Now operator overload");
        Account a3 = a1 + a2;
        a3.display();
        Console.ReadKey();
    }
}
```

========== sealed class ==========

Sealed class is used to restrict the inheritance features of object-oriented programming.

```
sealed class Test
{
   public void display()
   {
      Console.WriteLine("This is base class for seladed");
   }
}

//class newTest : Test   // throw error as the test class is shielded we cannt inherite it
//{

//}

class Program
{
   public static void Main(String[] args)
   {
      Test t = new Test();
      t.display();
   }
}
```

==========Boxing and unboxing=========

========== Boxing ==========

Boxing is process of converting a value type to reference type.

Eg.

Int, float, character (value type) ----------> object (reference type)

```
class Program
{
   public static void Main(String[] args)
   {
      int x = 10;
      Object a = x;  // object is a keyword we must write it
```

```
        x = 40;
        Console.WriteLine("Calling value of x   " + x);
        Console.WriteLine("Call obejct   " + a);
        Console.ReadKey();
    }
}
```

========== Unboxing ==========

It is process of converting reference type to value type is known as UNBOXING.

```
class program
{
    public static void main(String[] args)
    {
        int x = 10;
        Object a = x;  // object is a keyword we must write it . Boxing
        x = 40;
        int b = (int)a;  // unboxing
        Console.WriteLine("Calling value of x   " + x);
        Console.WriteLine("Call obejct   " + a);
        Console.WriteLine("after unbox   " + b); // as we unbox the object so the object value will now become int value
        Console.ReadKey();
    }
}
```

========== Generic type ==========

Generic allow you to write a class or method that can work with any data type. The detailed specification for each collection is found under the System.Collection.Generic namespace.

A generic method is declare with generic type parameter list enclosed within angular brackets.

========== Generic class ==========

Generic class syntax → class Name<T>

```
class Box<T>
{
```

```
    public T value;
    public Box(T value)
    {
        this.value = value;
    }

    public void display()
    {
        Console.WriteLine("Value = " + value);
        Console.WriteLine("Value type " + value.GetType());
    }
}

class Program
{
    public static void Main(String[] args)
    {
        Box<int> intBox = new Box<int>(100);
        Box<string> strngBox = new Box<string>("Bikram Gyawali");
        Box<double> doubleBox = new Box<double>(22.2);

        intBox.display();
        strngBox.display();
        doubleBox.display();
    }
}
```

========== Generic Method =========

Generic method syntax → void Method<T>(T param)

Example to show simple generic method and the swap algorithm implemetation

```
using System.Collections.Generic;
class Program
{
    static  void  display <T>(T value)
    {
```

```
        Console.WriteLine("Value = "+value);
        Console.WriteLine("Value of type = " + value.GetType());
    }


    // for implementing swap method

    public static  void swap<T>(ref T a , ref T b)
    {
        T temp = a;
        a = b;
        b =temp;
    }

    public static void Main(String[] args)
    {
        Console.WriteLine("---- Display Method -----");
        display<int>(55);   // method is static so we dont need to create instance
        display<string>("Bikram Gyawali");
        display<double>(554.5);

        Console.WriteLine("----- int value   swap -------");
        int x = 20, y = 30;
        Console.WriteLine("Before swap  x ={0}  y={1}", x, y);

        swap<int>(ref x, ref y);
        Console.WriteLine("After swap  x ={0}  y={1}", x, y);

        Console.WriteLine("----- string value   swap -------");
        string s1 = "Bikram", s2 = "Hari";
        Console.WriteLine("Before swap  s1 ={0}  s2={1}", s1, s2);

        swap<string>(ref s1, ref s2);
        Console.WriteLine("Afer swap  s1 ={0}  s2={1}", s1, s2);

        Console.ReadKey();
    }
}
```

========== Queue ==========

Enqueue → add element at rear

Dequeue → remove element from front

Peek → see front element without removing

Count → total elements

Contains → check if element exists

WAP to implemet all queue operation

```
using System;
using System.Collections.Generic;

class Program
{
   public static void Main(String[] args)
   {
      Queue<int> qu = new Queue<int>();

      Console.WriteLine("Implement Enqeue");
         qu.Enqueue(1);
         qu.Enqueue(20);
         qu.Enqueue(3);
         qu.Enqueue(50);
         qu.Enqueue(1);

         Console.WriteLine("Display queue value");

         foreach (int a in qu)
         {
            Console.WriteLine(a);
         }
         Console.WriteLine("Total Number of items in queue" + qu.Count());
         Console.WriteLine("Pick the first value of the queue = " + qu.Peek());
```

```
        Console.WriteLine("Implement Deqeue");
        Console.WriteLine("Removed Value = " +qu.Dequeue());
        Console.WriteLine("Removed Value = " +qu.Dequeue());
        Console.WriteLine(" After Implement Deqeue");

        foreach(int b in qu)
        {
           Console.WriteLine(b);
        }

        Console.WriteLine("Does queue contains 30 " + qu.Contains(30));
        Console.WriteLine("Does queue contains 50 " + qu.Contains(50));

        Console.ReadKey();


    }
}
```

========== Stack ==========

Push → add element on top

Pop → remove element from top

Peek → see top element without removing

Count → total elements

Contains → check if element exists

WAP to implement stack

```
using System;
using System.Collections.Generic;

class Program
{
   public static void Main(String[] args)
   {
      Stack<string> st = new Stack<string>();
```

```
        st.Push("Bikram");
        st.Push("Hari");
        st.Push("Ram");
        st.Push("Shyam");

        Console.WriteLine("items in stack");

        foreach (string a in st) Console.WriteLine(a);

        Console.WriteLine("Total items in stack " + st.Count());
        Console.WriteLine("First items in stack " + st.Peek());

        Console.WriteLine("Removed : " +st.Pop());
        Console.WriteLine("Removed : " +st.Pop());

        Console.WriteLine("items in stack after pop");

        foreach (string a in st) Console.WriteLine(a);

        Console.WriteLine("Stack Contain Bikram " + st.Contains("Bikram"));
        Console.WriteLine("Stack Contain Shyam  " + st.Contains("Shyam"));

        Console.ReadKey();
    }
}
```

== wap to implement undo implemenation

```
using System;
using System.Collections.Generic;

class Program
{
    public static void Main(String[] args)
    {
        Stack <string> st = new Stack<string>();
```

```
        Console.WriteLine("== Implement user actions== \n");
        st.Push("Typed Hello");
        Console.WriteLine("Action: Typed Hello");
        st.Push("Typed hehe");
        Console.WriteLine("Action: Typed hehe");

        Console.WriteLine("== Undo user actions== \n");

        while (st.Count > 0)
        {
            string action = st.Pop();
            Console.WriteLine("Undo : "+action);
        }
        Console.WriteLine("\nNothing left to undo");
        Console.ReadKey();
    }
}
```

========== Link List =======

AddFirst → add at beginning

AddLast → add at end

AddBefore → add before a node

AddAfter → add after a node

Remove → remove a node

RemoveFirst → remove first node

RemoveLast → remove last node

Contains → check if value exists

```
using System;
using System.Collections.Generic;
class Program
{
    public static void Main(string[] args)
    {
        LinkedList<int> list = new LinkedList<int>();

        Console.WriteLine("Add value in linkedlist");
```

```
list.AddLast(20);
list.AddLast(30);
list.AddLast(50);
list.AddLast(10);
list.AddLast(90);
list.AddLast(80);

Console.WriteLine("Elements in linkedlist");
foreach (int items in list) Console.Write(items + " ");

Console.WriteLine("\n First item : " + list.First.Value);
Console.WriteLine("\n Last item : " + list.Last.Value);
Console.WriteLine("\n Total item : " + list.Count);

Console.WriteLine("== AddBefore and AddAfter ==");

Console.WriteLine("Added Element 5 before 20");
LinkedListNode<int> node =list.Find(20);

list.AddBefore(node,5);
Console.WriteLine("Added Element 35 after 30");
LinkedListNode<int> node2 = list.Find(30);

list.AddAfter(node2, 35);

Console.WriteLine("\n After Addition");
foreach (int items in list) Console.Write(items + " ");


Console.WriteLine("\n Remove");

list.Remove(10);
Console.WriteLine("Remove 10");
list.RemoveFirst();
Console.WriteLine("Remove first item");
list.RemoveLast();
Console.WriteLine("Remove last item");
```

```
        Console.WriteLine("\n After Remove");
        foreach (int items in list) Console.Write(items + " ");

        Console.WriteLine("\n ==Contain ==");

        Console.WriteLine("Stack contains 5 ? " + list.Contains(5));
        Console.WriteLine("Stack contains 35 ? " + list.Contains(35));
        Console.ReadKey();

    }
}
```

========== Chapter 4 advance c# ====

========== Delegate =======

syntax: <access modifier> delegate <return type> <delegate name>(parameter);

* delegate and the method has the same signature

came under System.Delegate class

##Wap to implement a delegate example program

```
using System;
class Program
{
        public delegate void DelegateProgram(string text);
        public static void upper(string a)
        {
            Console.WriteLine("Text in upper case " + a.ToUpper());
        }
        public static void lower(string b)
        {
            Console.WriteLine("Text in lower case " + b.ToLower());
        }
```

```
    public static void Main(String[] args)
    {
        DelegateProgram d;
        d = new DelegateProgram(upper);
        Console.WriteLine("--Upper --");
        d("Hello hehe");
        d = new DelegateProgram(lower);
        Console.WriteLine("--Lower --");
        d("Hello Bikram");
        Console.ReadKey();
    }
}
```

WAP to find square of number using delegate

```
using System;

class Program
{
    public delegate int DelegateSquare(int n);

    public static int square(int n)
    {
        return n* n;
    }
    public static void Main(String[] args)
    {
        DelegateSquare de = new DelegateSquare(square);
        Console.WriteLine("The square of number is " + de(3));
        Console.ReadKey();
    }
}
```

wap to implement a simple calculator using the delegate

```
using System;
class program
{
   public delegate int DelegateCal(int a, int b);

   public static int sum(int a, int b) { return a + b; }
   public static int diff(int a, int b) { return a - b; }
   public static int mul(int a, int b) { return a * b; }

   public static void Main(String[] args)
   {
      DelegateCal d;
      d = new DelegateCal(sum);
      Console.WriteLine("The sum result " + d(5, 7));
      d = new DelegateCal(diff);
      Console.WriteLine("The diff result " + d(50, 7));
      d = new DelegateCal(mul);
      Console.WriteLine("The mul result " + d(7, 7));
      Console.ReadKey();

   }
}
```

=== multidelegate ====

```
using System;

class program
{
   public delegate void Calculate(int a, int b);

   public static void Add(int a , int b)
   {
      Console.WriteLine("The sum is " + (a + b));
   }
   public static void Sub(int a, int b)
   {
```

```
        Console.WriteLine("The difference is " + (a - b));
    }

    public static void Mul(int a,int b)
    {
        Console.WriteLine("The mul is " + (a * b));
    }

   public static void Main(String[] args)
    {
        Calculate cl = new Calculate(Add);
        cl += Sub;
        cl += Mul;
        cl(20, 30);
        Console.ReadKey();
    }
}
```

Imagine a student submits an exam form. After the submission, the system performs several actions automatically. The student's information is first saved in the database to record the application. Then, an SMS is sent to the student confirming that the form has been successfully submitted. At the same time, an email notification is also sent with the same confirmation details.

```
class Program
{
    public delegate void Student();

    public static void SMS()
    {
        Console.WriteLine("Send message");

    }

    public static void Email(){
        Console.WriteLine("Email message");
    }
```

```
    public static void SaveRecord()
    {
        Console.WriteLine("Record save to database");
    }

    public static void Main(String[] args)
    {
        Student s = new Student(SMS);
        s += Email;
        s += SaveRecord;
        //s() invoke delegate
        s.Invoke(); // invoke delegate



        // revoke the delegate
        Console.WriteLine("\n\nafter Reinvoke the email is removed\n\n");
        s -= Email;
        s.Invoke();

        Console.ReadKey();
    }
}
```

====== Event in C# =====

Defining an event is a two-step process.

❖ First you need to defining a delegate type that will hold the list of methods to be called when the event is fired.

❖ Next, you declare an event using the event keyword.

how event work

Step 1 → declare a delegate

Step 2 → declare an event using that delegate

Step 3 → subscribe to the event using +=

Step 4 → fire the event when something happens

wap to implement a event inside a single class

```
class program
{
    public  delegate void DelegateExp();
    public static event DelegateExp EventExp;  // we should make it static if we define it inside the main class

    public  static void eventClick()
    {
        Console.WriteLine("Event Clicked");
        if(EventExp != null)
        {
            EventExp();
        }
        else
        {
            Console.WriteLine("No subscriber");
        }
    }

    static void ChangeMessage()
    {
        Console.WriteLine("The event change the message");
    }

    static void ChangeColor()
    {
        Console.WriteLine("The background is change");
    }

    public static void Main(String[] args)
    {
        eventClick(); // this show no subscriber
        EventExp += ChangeColor;
        EventExp += ChangeMessage;
        EventExp.Invoke();
    }
}
```

implement event in 2 different class

```
class Example
{
    public delegate void ExampleDelegate();
    public ExampleDelegate del;  // this is delegate
    public event ExampleDelegate onClick;  // onclick is just a variable we can replace it with other names
    public void clickEvent()
    {
        Console.WriteLine("The event is clicked");
        if (onClick != null) onClick();
    }
}
class program
{
    static void ChangeMessage()
    {
        Console.WriteLine("Change the message");
    }

    static void ChangeColor()
    {
        Console.WriteLine("Change color");
    }

    public static void Main(String[] args)
    {

        Example e = new Example();

        //for delegate
        Console.WriteLine("----Call from the delegate -----");
        e.del += ChangeMessage;
        e.del();  // we can call delegate from outside of the class

        //for event
```

```
        Console.WriteLine("----Call from the event -----");
        e.onClick += ChangeMessage;
        e.onClick += ChangeColor;

        //e.onClick(); it will throw error as we cannt call event outside the class so we call the method

        e.clickEvent();

        Console.WriteLine("---after Reinvoke in event----");
        e.onClick-= ChangeMessage;
        e.clickEvent();
        Console.ReadKey();
    }
}
```

==Anomous function/method ====

Syntax:

```
delegate  (parameter_list)
{
//code here
};
```

wap to add any two numbers using the anomous function

```
using System;

class ProgramP
{
    public delegate int Calculate(int a, int b);

    public static void Main(String[] args)
    {
        Calculate add = delegate (int a, int b)
        {
            return a + b;
```

```
        };
        Console.WriteLine("The sum ="+ add(2, 3));
        Console.WriteLine("The sum ="+ add(20, 3));
        Console.ReadKey();
    }
}
```

====== Lambda====

Expression Lambda: Consists of the input and the expression.

Syntax:

Input=>expression;

❖ Statement Lambda: Consists of the input and a set of statements to be executed. It can be used along the delegates.

Syntax:

Input =>{statements};

wap a simple program to show the use of the lambda expression

```
using System;
class Program
{
    public static int test() => 20;
    public static int sum(int x) => x + 20;

    public static void Main(String[] args)
    {
        int x = test();
        Console.WriteLine("The result is " + sum(x));
        Console.ReadKey();
    }

}
```

use the lambda express to sum 2 number and show your name in upppercase

```
class Program
{
    delegate int Sum(int x, int y);
    delegate  string Name(string name);
    delegate void  Address(string address);
    public static void Main(String[] args)
    {
        Sum add = (x, y) => x + y;
        Name upperCase = (name) =>  name.ToUpper();
        Address a = address => Console.WriteLine("You live in " + address);

        // for sum
        Console.WriteLine("Enter first number");
        int n1 = Convert.ToInt16(Console.ReadLine());
        Console.WriteLine("Enter second number");
        int n2 = int.Parse(Console.ReadLine());

        Console.WriteLine("Sum of fix digit " + add(2, 3));
        Console.WriteLine("Sum is of user input " + add(n1, n2));
        // for upper case
        Console.WriteLine("Enter your name");
        string uname = Console.ReadLine();
        Console.WriteLine("{0} in uppercase is = {1}", uname, upperCase(uname));

        Console.WriteLine("User predefine address as kathmandu");

        a("Kathmandu");
        Console.ReadKey();

    }
}
```

====== Exception Handling =====

wap to implement dividebyzero exception

```
class program
{
 public static void Main(String[] args)
   {
      Console.WriteLine("Enter first number");
      int n1 =int.Parse(Console.ReadLine());
      Console.WriteLine("Enter second number");
      int n2 = int.Parse(Console.ReadLine()); // we can use Conver.ToInt32 or other it will not throw error
      try
      {
         int result = n1 / n2;
         Console.WriteLine("The result is " + result);
      }catch(DivideByZeroException ex)
      {
         Console.WriteLine("Enter 2nd number greater than 0 " );
         Console.WriteLine(ex.Message);
      }
      catch (Exception e)  // this will work for every type of expection
      {
         Console.WriteLine(e.Message);
      }
      finally
      {
         Console.WriteLine("Run everytime");
      }
      Console.ReadKey  ();
   }
}
```

========== implement the throw exception ======

```
class Program
{
   public static void Main(String[] args)
   {
```

```
        try
        {
            Console.WriteLine("Enter your age");
            int age = int.Parse(Console.ReadLine());

            if(age<0 || age > 20)
            {
                throw new Exception("Enter a valid age");
            }
            Console.WriteLine("Your age is " + age);
        } catch(Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
        Console.ReadKey();
    }
}
```

======== LINQ (Language integrated query) ========
System.Linq provides:
Select → pick specific property from each item
Where → filter items by condition
OrderBy → sort items
Count → count items
First → get first item
Any → check if any item matches

System.Collections.Generic provides:
List<T> → the list itself

WAP to implement lambda in linq to find the even number from a list

```
using System;
using System.Collections.Generic;

class Program
{
   // if we dont use lambda then we have to do like this
  //public static bool isEven(int n)
  // {
  //     return n %2==0;
  // }
    public static void Main(String[] args)
    {
        List<int> numbers = new List<int>() { 3, 2, 5, 16, 77, 32 };
        var result = numbers.FindAll(x => x % 2 == 0);  // var will determine the datatype automatically
List<int> result = numbers.FindAll(x => x % 2 == 0);  // we can use this line insted of the var
        //List<int> result = numbers.FindAll(isEven);  this is for without lambda
```

```
        Console.WriteLine("List of even number");
        foreach (int n in result) Console.Write("\t" + n);

    Console.ReadKey();
    }
}
```

WAP to display names of all animals from a list and arrange in ascending or descending using LINQ and lambda.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Animal
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    public static void Main(String[] args)
    {
var animal = new Animal[]{  // can use this method
new Animal{Name="Horse",Age=22},
        new Animal{Name="Dog" , Age=8},
        new Animal{Name="Cat",Age=3}

};

        List<Animal> animal = new List<Animal>()// we pass the object in the list as Animal
        {

        new Animal{Name="Horse",Age=22},
        new Animal{Name="Dog" , Age=8},
        new Animal{Name="Cat",Age=3}

        };

        var showName = animal.Select(a => a.Name); // the var will automatically take the value of datatype

        // to display the name

        foreach (var name in showName) Console.WriteLine("\t" + name);

        // for descending

        //var age = animal.OrderByDescending(a => a.Age);  // for descending
        var age = animal.OrderBy(a => a.Age);  // for ascending
        foreach (var a in age)
        {
            Console.WriteLine("Animal {0} age = {1}" , a.Name, a.Age);
                };
        Console.ReadKey();

    }
}
```

WAP to implement where operation of linq
Where() is used to filter data based on a condition.

```
using System;
using System.Collections.Generic;
```

```
using System.Linq;

class Program
{
   public static void Main(String[] args)
   {
      List<int> numbers = new List<int>() { 3, 24, 25, 23, 2 };
      var numbers = new List<int>() { 3, 24, 25, 23, 2 };
      var evenNumber = numbers.Where(x => x % 2 == 0);
      foreach (int x in evenNumber) Console.Write("\t"+x);
      Console.ReadKey();
   }
}
```

WAP to implement Join Operation in Linq
Join() combines two lists based on a common key.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Order
{
   public int Id { get; set; }
   public int CustomerId {  get; set; }
   public string ProductName {  get; set; }
}

class Customer
{
   public int Id { get; set; }
   public String Name { get; set; }
}

class Program
{
   public static void Main(String[] args)
   {
      List<Order> orders = new List<Order>()
      {
         new Order{Id=1, CustomerId=1,ProductName="Mac book"},
         new Order{Id=2, CustomerId=1,ProductName="lenevo book"},
         new Order{Id=3, CustomerId=2,ProductName="Iphone"},
         new Order{Id=4, CustomerId=3,ProductName="Mac book"},

      };

      var customers = new Customer[]  // we can use this method also of array
      {
         new Customer{Id=1,Name="Vikram"},
         new Customer{Id=2,Name="Hari"},
         new Customer {Id=3,Name="Sankalpa"}
      };

      var result = from c in customers
                   join o in orders
                   on c.Id equals o.CustomerId
                   select new { c.Id, c.Name, o.ProductName };

      foreach(var res in result)
      {
         Console.WriteLine("Customer id = {0} Customer Name = {1} buys" +
            " Product Name = {2}", res.Id, res.Name, res.ProductName);
      }
      Console.ReadKey();
```

```
    }
}
```

WAP to implement concat in linq
Concat() combines two lists and keeps duplicates.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    public static void Main(String[] args)
    {
        var number1 = new List<int>() { 3, 24, 2, 45, 2 }; // can use var and List<int> both
        List<int> number2 = new List<int>() { 5,64,2,9,6 };

        var result = number1.Concat(number2);

        foreach(var res in result)
        {
            Console.WriteLine(res);
        }
        Console.ReadKey();
    }
}
```

WAP to implement union operation in linq
Union() lists and remove duplicate

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    public static void Main(String[] args)
    {
        var number1 = new List<int>() { 3, 24, 2, 45, 2 }; // can use var and List<int> both
        List<int> number2 = new List<int>() { 5, 64, 2, 9, 6 };

        var result = number1.Union(number2);

        foreach (var res in result)
        {
            Console.WriteLine(res);
        }
        Console.ReadKey();
    }
}
```

WAP to implement GroupBy operation in linq group students according to their department
GroupBy() groups similar data together.

```
using System;
using System.Collections.Generic;
using System.Linq;

class Student
{
    public string Name { get; set; }
    public string DepartmentName { get; set; }
}
```

```
class Program
{
   public static void Main(String[] args)
   {
      var student = new Student[]
      {
         new Student{Name="Bikram",DepartmentName="BCA"},
         new Student{Name="Hari",DepartmentName="BCA"},
         new Student{Name="Ram",DepartmentName="BBA"},
         new Student{Name="Sita",DepartmentName="BBA"},
      };

      var groups = student.GroupBy(x => x.DepartmentName);

      foreach(var group in groups)
      {
         Console.WriteLine("Department: " + group.Key);
         foreach ( var sname in group)
         {
            Console.WriteLine(sname.Name);
         }
      }
      Console.ReadKey();
   }
}

=== code for asp web form ===

Syntax of asp follow Pascal case (eg :  TextBox)
 == for label==
<asp:Label ID="label1" runat="server" Text:"Hello"> </asp:label>

==for textbox===
<asp:TextBox ID="textbox" runat="server"> </asp:TextBox>

===for dropdownlist ====
<asp:DropDownList ID="ddp" runat="server">
<asp:ListItem Text="color" Value="red" />
<asp:ListItem Text="green" Value="Green"/>
</asp:DropDownList>


==== for radio button ===
<asp:RadioButton ID="rd1" Text="Male" GroupBy="Gender" Value="Male" runat="server"/>

==for checkbox ===
<asp:CheckBox ID="cb1" Text="Agrees terms"  runat="server" />

==for button ==
<asp:Button ID="btn1" Text="Submit" runat="server" OnClick="btn_submit" />

== for comparevalidator ==
ControlToValidate  →  the control being validated
ControlToCompare  →  the control to compare against
Operator  →  Equal, NotEqual, GreaterThan, LessThan etc
Type  →  String, Integer, Double, Date, Currency
ErrorMessage  →  message shown when validation fails
Display  →  Static, Dynamic, None


*/
```

```
using System;
enum Day
{
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday,
    Sunday

}

class Program
{
    public static void Main(String[] args)
    {
        Day today = Day.Monday;  // we use day not string or int the name of the enum is used
        Console.WriteLine("Today day is "+ today);
    }
}
```

========== Create web form with proper validation and the styling ==========

<!--
Write an ASP.NET program to create a Student Registration Form that includes TextBox,
RadioButton, CheckBox, DropDownList and demonstrates the use of RequiredFieldValidator,
CompareValidator, RangeValidator, RegularExpressionValidator and ValidationSummary without database connection. -->

In exam we can directly start with the form tag or html tag no need to remember whole structure

Register.aspx

```
<%@ Page Language="C#"
   AutoEventWireup="true"
   CodeBehind="Register.aspx.cs"
   Inherits="WebApplication.Register" %>
<!DOCTYPE html>
<html>

   <head runat="server">
      <title> Student Register</title>
      <style>
         body{
            font-family:Arial ;
            margin:20px;
         }
         table{
            border-collapse:collapse;
            width:600px
         }
         td{
            padding:8px;
         }
         .head{
            background-color: #4CAF50;
            color: white;
            text-align: center;
            font-size: 20px;
            padding: 10px;
```

```
        }
        .sucess{
            color:green;
            font:14px;


        }
    </style>
</head>
<body>
    <form id="form1" runat="server">
        <table border="1">
            <tr>
                <td colspan="3">
                    Student Register System
                </td>
            </tr>
            <!-- Name -->

            <tr>
                <td>
                    Full Name:
                </td>
                <td>
                    <asp:TextBox ID="txtName" runat="server" Width="200px"/>
                </td>
                <td>
                    <asp:RequiredFieldValidator ID="rfvName"
                        runat="server" ControlToValidate="txtName"
                        ErrorMessage="Name is Required" Text="*" ForeColor="Red" ValidationGroup="reg"
                        />
                </td>
            </tr>

             <!-- Email
                 ValidationExpression="^[^\s@]+@[^\s@]+\.[^\s@]+$"
                -->

        <tr>
            <td>
                Email:
            </td>
            <td>
                <asp:TextBox ID="txtEmail" runat="server" Width="200px"/>
            </td>
            <td>
              <asp:RequiredFieldValidator ID="rfvEmail"
                runat="server" ControlToValidate="txtEmail"
                ErrorMessage="Email field is Required" Text="*" ForeColor="Red"
                ValidationGroup="reg"
                />

             <asp:RegularExpressionValidator ID="revEmail"
                runat="server" ControlToValidate="txtEmail" ErrorMessage="Invalid Email format" Text="*" ForeColor="Red"
                ValidationExpression="^[^\s@]+@[^\s@]+\.[^\s@]+$" ValidationGroup="reg"
                />
            </td>
        </tr>
                    <!-- phone   -->

        <tr>
            <td>
                Phone Number:
            </td>
            <td>
                <asp:TextBox ID="txtPhone" runat="server" Width="200px"/>
```

```
        </td>
        <td>
          <asp:RequiredFieldValidator
            ID="rfvPhone" ControlToValidate="txtPhone"
            ErrorMessage="Phone number is required" Text="*" ForeColor="Red" runat="server"
            ValdiationGroup="reg"
            />

          <asp:RegularExpressionValidator ID="revPhone" runat="server" ControlToValidate="txtPhone"
            ErrorMessage="Phone number must be 10 digits" Text="*" ForeColor="Red"
            ValidationExpression="[0-9]{10}" ValidationGroup="reg"
            />

        </td>
      </tr>


                    <!-- Age implements RangeValidator -->

<tr>
  <td>
    Age:
  </td>
 <td>
   <asp:TextBox ID="txtAge" Width="200px"  runat="server"/>
 </td>
  <td>
   <asp:RequiredFieldValidator runat="server" ID="rfvAge" ControlToValidate="txtAge" ErrorMessage="Age is Required !"
     Text="*" ForeColor="Red" ValidationGroup="reg"/>

    <asp:RangeValidator runat="server" ID="rvAge" ControlToValidate="txtAge"
      MinimumValue="16" MaximumValue="40" Type="Integer"
      ErrorMessage="Age must be 16 to 40 "  Text="*" ForeColor="Red" ValidationGroup="reg"
       />

  </td>
</tr>
          <!-- Password -->
          <tr>
            <td>
              Password :
            </td>
            <td>
              <asp:TextBox runat="server" Width="200px" ID="txtPassword" TextMode="Password"/>
            </td>
            <td>
              <asp:RequiredFieldValidator runat="server" ID="rfvPassword"
                ControlToValidate="txtPassword" ErrorMessage="Password is Required"
                Text="*" ForeColor="Red" ValidationGroup="reg"
                />
            </td>
          </tr>

          <!--Confirm Password implement CompareValidator -->
<tr>
  <td>
   Confirm  Password :
  </td>
  <td>
    <asp:TextBox runat="server" Width="200px" ID="txtConfirm" TextMode="Password"/>
  </td>
  <td>
    <asp:RequiredFieldValidator runat="server" ID="rfvConfirm"
      ControlToValidate="txtConfirm" ErrorMessage=" Confirm Password is Required"
      Text="*" ForeColor="Red" ValidationGroup="reg"
      />
```

```
      <asp:CompareValidator runat="server" ID="cvConfirm" ValidationGroup="reg"
        ControlToValidate="txtConfirm" ControlToCompare="txtPassword" Operator="Equal" Type="String"
        ErrorMessage="Passwords do no match" Text="*" ForeColor="Red"

        />
    </td>
</tr>

            <!-- Gender implement RadioButton-->

            <tr>
              <td>
                Gender :
              </td>
              <td>
                <asp:RadioButton ID="rbMale" runat="server" Text="Male" GroupName="Gender"  />
                <asp:RadioButton  ID="rbFemale" runat="server" Text="Female" GroupName="Gender"/>
                <asp:RadioButton ID="rbOther" runat="server" Text="Other" GroupName="Gender" />
              </td>
            </tr>

            <!-- Course implement drop down -->
            <tr>
              <td>
                Courses :
              </td>
              <td>
                <asp:DropDownList ID="ddlCourse" runat="server" Width="200px" >
                  <asp:ListItem Value="">Select Course</asp:ListItem>
                  <asp:ListItem Value="BCA">BCA</asp:ListItem>
                  <asp:ListItem Value="BBA">BBA</asp:ListItem>
                  <asp:ListItem Value="BHM">BHM</asp:ListItem>
                </asp:DropDownList>
              </td>
              <td>
                <asp:RequiredFieldValidator ID="rfvCourse" runat="server" ValidationGroup="reg"
                  ControlToValidate="ddlCourse" InitialValue="" ErrorMessage="Select a Course" Text="*" ForeColor="Red"
                  />
              </td>
            </tr>

            <!-- Subject implement CheckBox -->

            <tr>
              <td>
                Subjects:
              </td>
              <td>
                <asp:CheckBox ID="chkMath" runat="server" Text="Math" /> <br />
                <asp:CheckBox ID="chkScience" runat="server" Text="Science"/> <br />
                <asp:CheckBox ID="chkEnglish" runat="server" Text="English"/>
            <br/>
            <asp:CheckBox ID="chkComputer" runat="server" Text="Computer"/>
              </td>
              <td></td>
            </tr>

            <!-- Address  TextBox with mode -->
            <tr>
              <td>
                Address :
              </td>
              <td>
                <asp:TextBox ID="txtAddress" runat="server" TextMode="MultiLine" Rows="3" Width="200px" />
              </td>
              <td>
```

```
            <asp:RequiredFieldValidator ID="rfvAddress" runat="server"   ValidationGroup="reg"
               ControlToValidate="txtAddress"
               ErrorMessage="Address is required!"
               ForeColor="Red"
              />
          </td>
            </tr>

            <!-- Button
               CausesValidation=false skip validation when the button click-->
            <tr>
               <td colspan="3" align="center">
                  <asp:Button ID="btnSubmit" runat="server" OnClick="btnSumbit_Click" Text="Register"
                     ValidationGroup="reg"/>
                  <asp:Button ID="btnReset" runat="server" OnClick="btnReset_Click" Text="Reset" ValidationGroup="reg"
                     CausesValidation="false"/>
               </td>
            </tr>

            <!-- Result asp label -->
            <tr>
               <td colspan="3" align="center">
                  <asp:Label ID="lblMessage" runat="server" CssClass="sucess" />
               </td>
            </tr>

            <!-- Validation Summary -->
            <asp:ValidationSummary runat="server" ID="vsSummary" HeaderText="Please fix these errors:" ForeColor="Red"
               ValidationGroup="reg"/>
         </table>
      </form>
   </body>
</html>
```

======== **Register.aspx.cs file** ==========

```
using System;
using System.Web.UI;

namespace WebApplication
{
  public partial class Register : Page
   {
      protected void Page_Load (Object sender,EventArgs e)
      {

      }

      protected void btnSumbit_Click(Object sender,EventArgs e)
      {
         if (Page.IsValid)
         {
            string gender = "";
            if (rbMale.Checked) gender = "Male";
            else if (rbFemale.Checked) gender = "Female";
            else if (rbOther.Checked) gender = "Other";
            else gender = "Not Selected";

            string subjects = "";
            if (chkMath.Checked) subjects += "Math";
            if (chkEnglish.Checked) subjects += "English";
            if (chkScience.Checked) subjects += "Science";
```

```
            if (chkComputer.Checked) subjects += "Computer";

            lblMessage.Text = "<b> Registration of Student Successful  </b> <br/> <br/>" +
               "Name :      " + txtName.Text + "<br/>" +
               "EMail :    " + txtEmail.Text + "<br/>" +
               "Age :      " + txtAge.Text + "<br/>" +
               "Address :  " + txtAddress.Text + "<br/>" +
               "Phone :    " + txtPhone.Text + "<br/>" +
               "Gender :    " + gender + "<br/>" +
               "Course :   " + ddlCourse.SelectedValue + "<br/>" +
               "Subjects :  " + subjects;


         }
      }

      protected void btnReset_Click(Object senders,EventArgs e)
      {
         txtName.Text = "";
         txtAge.Text = "";
         txtAddress.Text = "";
         txtPhone.Text = "";
         txtEmail.Text = "";
         txtPassword.Text = "";
         txtConfirm.Text = "";

         rbMale.Checked = false;
         rbFemale.Checked = false;
         rbOther.Checked = false;

         chkComputer.Checked = false;
         chkEnglish.Checked = false;
         chkMath.Checked = false;
         chkScience.Checked = false;

         ddlCourse.SelectedIndex = 0;
         lblMessage.Text = "";
      }
   }
}
```

Please fix these errors:

- Invalid Email format
- Phone number must be 10 digits
- Age must be 16 to 40
- Passwords do no match
- Address is required!

Student Register System		
Full Name:	Bikram Gyawali	
Email:	dd	*
Phone Number:	33342	*
Age:	444	*
Password :	••	
Confirm Password :	••••	*
Gender :	○ Male ○ Female ○ Other	
Courses :	BCA	
Subjects:	☐ Math ☐ Science ☐ English ☐ Computer	
Address :		Address is required!
Register Reset		

Student Register System		
Full Name:	Bikram Gyawali	
Email:	aa@gmail.com	
Phone Number:	9785353535	
Age:	20	
Password :		
Confirm Password :		
Gender :	◉ Male ○ Female ○ Other	
Courses :	BCA	
Subjects:	☑ Math ☑ Science ☐ English ☑ Computer	
Address :	Kathmandu	
Register Reset		

Registration of Student Successful

Name : Bikram Gyawali
EMail : aa@gmail.com
Age : 20
Address : Kathmandu
Phone : 9785353535
Gender : Male
Course : BCA
Subjects : MathScienceComputer

==== Program to create a calculator ===

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Calculator.aspx.cs" Inherits="WebApplication.Calculator"
%>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
    <style>

  body{
      display:flex;
      justify-content:center;
      margin-top:50px;
      background-color:aqua;
  }
  form{
      padding:20px;
      width:500px;
      border:1px solid #000;
      border-radius:8px;
      box-shadow: 0 0 10px rgb(0,4,0,0.1)

  }

        .caldiv{
    display:flex;
    align-items:center;
    margin-bottom:20px;

}
        .caldiv h2{
            width:220px;
            font-size:18px;
        }
        .caldiv input{
            padding:8px 10px;
            font-size:16px;
            flex:1

        }
        .button_row{
            display:flex;
            justify-content:center;
            gap:12px;
            margin:25px 0;
        }
         .button_row input{
            padding:10px;
            color:White;
            background-color:Red;
                margin:3px;
                border:1px solid #ccc;
                gap:3px
        }
        .result_row{
           display:flex;
           justify-content:center;
           gap:12px;
        }
        .error{
            color:red;
            font-size:14px;
```

```
    }
  </style>
<body>
  <form id="form1" runat="server">
    <div class="caldiv">
    <h2>Enter your 1st number</h2>
      <asp:TextBox runat="server" ID="num1" />
      <asp:RequiredFieldValidator  runat="server" ID="rfvNum1" ValidationGroup="cal"
        ControlToValidate="num1" Text="*" ErrorMessage="Number is required in n1"
        CssClass="error"
        />
      <asp:CompareValidator runat="server" ID="cvNum1" ValidationGroup="cal"
        ControlToValidate="num1" Operator="DataTypeCheck" Type="Double"
        ErrorMessage="Entere a validate number" CssClass="error"

        />
    </div>
    <div class="caldiv">
      <h2>Enter your second number</h2>
      <asp:TextBox runat="server" ID="num2" />

      <asp:RequiredFieldValidator  runat="server" ID="rfvNum2" ValidationGroup="cal"
        ControlToValidate="num2" ErrorMessage="Number 2 is Required" Text="*" CssClass="error"

        />

      <asp:CompareValidator  runat="server" ID="cvNum2" ValidationGroup="cal"
        ControlToValidate="num2" Operator="DataTypeCheck" Type="Double"
        ErrorMessage="Invalid Number format in n2" CssClass="error"

        />
    </div>
    <div class="button_row">
      <asp:Button runat="server" ID="btn1" OnClick="btnAdd" Text="Add" ValidationGroup="cal"/>
      <asp:Button runat="server" ID="btn2" OnClick="btnSub" Text="Sub" ValidationGroup="cal"/>
      <asp:Button  runat="server" ID="btn3" OnClick="btnMul" Text="Mul" ValidationGroup="cal"/>
      <asp:Button runat="server" ID="btn4" OnClick="btnDiv" Text="Div" ValidationGroup="cal"/>
    </div>
    <div class="result_row">
      <asp:Label runat="server" ID="lbl">Result</asp:Label>
    <asp:Label runat="server" ID="result"/>
    </div>
    <asp:ValidationSummary HeaderText="Please solve these errors" ForeColor="Red" runat="server" ID="vsCal"
ValidationGroup="cal" />
  </form>
</body>
</html>
```

Calculator.aspx.cs

```
using System;

using System.Web.UI;

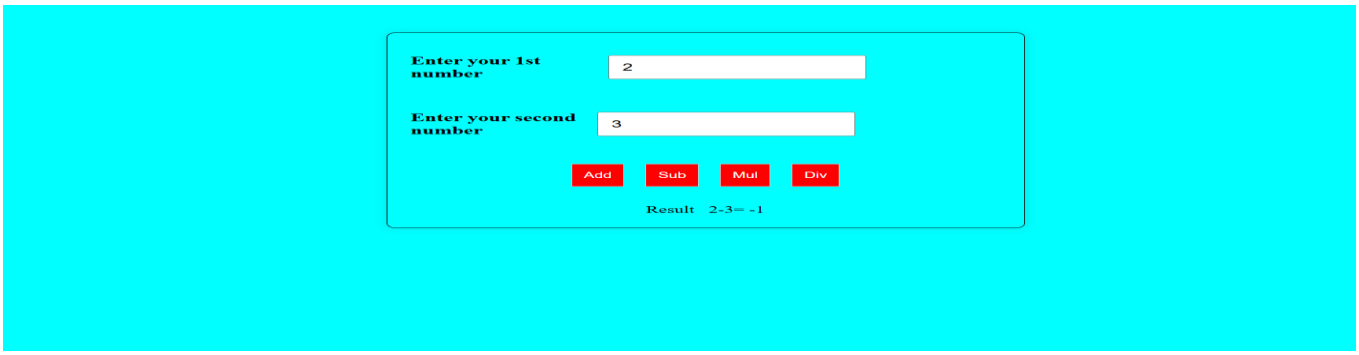

namespace WebApplication
{
  public partial class Calculator : Page
  {
    protected void Page_Load(Object sender,EventArgs e)
    {

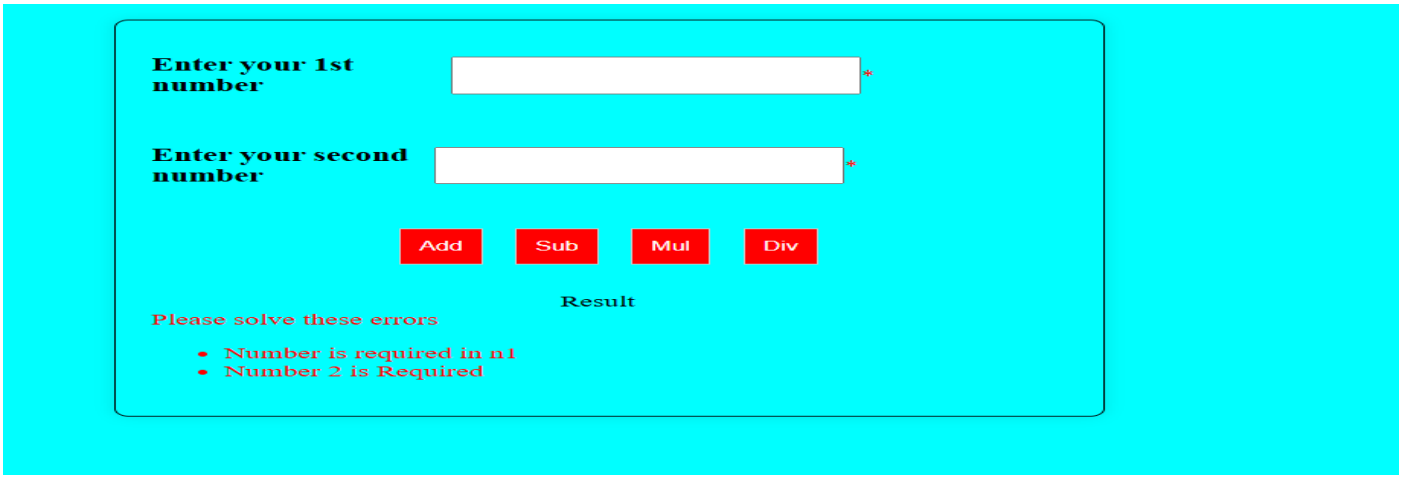
    }
    protected void btnAdd (Object sender,EventArgs e)
    {
      if (Page.IsValid)
```

```
        {
            double n1 = Convert.ToDouble(num1.Text);
            double n2 = Convert.ToDouble(num2.Text);
            result.Text = n1 + "+" + n2 + "= " + (n1 + n2).ToString();
        }
    }

    protected void btnSub(Object sender,EventArgs e)
    {
        if (Page.IsValid)
        {
            double n1 = Convert.ToDouble(num1.Text);
            double n2 = Convert.ToDouble(num2.Text);
            result.Text = n1 + "-" + n2 +  "= " +(n1 - n2).ToString();
        }
    }
    protected void btnMul(Object sender, EventArgs e)
    {
        if (Page.IsValid)
        {
            double n1 = Convert.ToDouble(num1.Text);
            double n2 = Convert.ToDouble(num2.Text);
            result.Text = n1 + "*" + n2 +  "= " +(n1 * n2).ToString();
        }
    }

    protected void btnDiv(Object sender,EventArgs e)
    {
        double n1 = Convert.ToDouble(num1.Text);
        double n2 = Convert.ToDouble(num2.Text);
        if (n2 == 0){
            result.Text = "Cannot divide by zero";
            return;
        }
        else
          {
            result.Text = n1 + "/" + n2 +  "= " +"= " +(n1 / n2).ToString();
        }
    }
  }
}
```





=== Program to take input and display in another page ===

Redirect.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Redirect.aspx.cs" Inherits="WebApplication.Redirect" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title></title>
</head>
<body style="background-color:deepskyblue">
    <form id="form1" runat="server">

        <div>
            <h1>Redirect form </h1>

            <h1> Enter Your Name : </h1>
            <asp:TextBox runat="server" ID="txt1" Width="300" />

            <h3>Contact :</h3>
            <asp:TextBox runat="server" ID="txt2"  Width="300"/>
            <br />   <br />
            <asp:Button  runat="server" ID="btn1" Text="Submit"  OnClick="btn_Clicked" Width="150"/>
        </div>

    </form>
</body>
</html>
```

Redirect.aspx.cs

```
using System;

using System.Web.UI;
```

```
namespace WebApplication
{
   public partial class Redirect : Page
   {
  public void Page_Load(Object sender,EventArgs e)
      {

      }
      protected void btn_Clicked(Object sender,EventArgs e)
      {
         string Name = txt1.Text;
         long Number = Convert.ToInt64(txt2.Text);  //long is used here because the number
                                          //size is longer so required long to store

         Response.Redirect("Display.aspx?name=" + Name + "&number=" + Number);
      }
   }
}
```

Display.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Display.aspx.cs" Inherits="WebApplication.Display" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
   <title></title>
</head>
<body style="background-color:navajowhite">

   <form id="form1" runat="server">
      <div>
         <h2>Received data</h2>

         <br /> <br />

         <h3>Name : </h3>
         <asp:Label ID="lbl1" runat="server" />

         <br /> <br />

         <h3>Conact Number</h3>

         <asp:Label ID="lbl2" runat="server" />
      </div>
   </form>
</body>
</html>
```

Display.aspx.cs

```
using System;
using System.Web.UI;

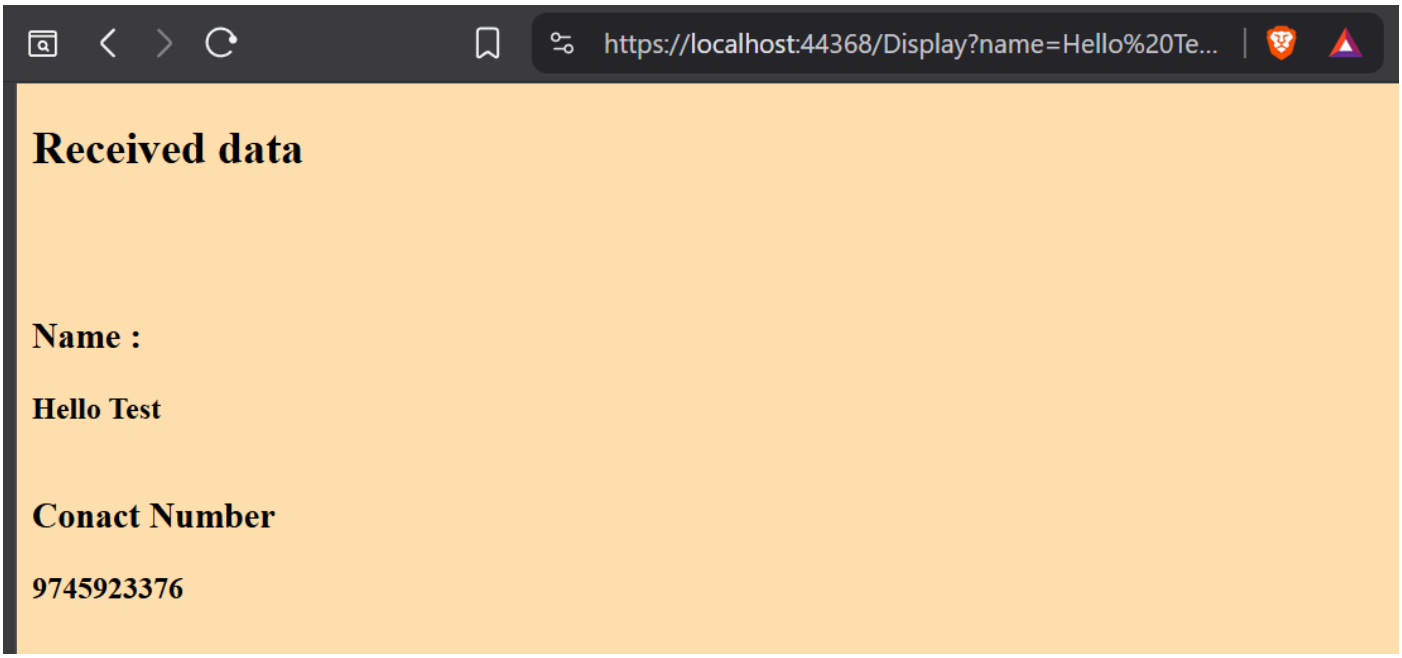
 namespace WebApplication
{
   public partial class Display : Page
```

```
    {
        public void Page_Load(Object sender,EventArgs e) {

            lbl1.Text = Request.QueryString["name"];
            lbl2.Text = Request.QueryString["number"];
        }

    }
}
```





===Program to connect database and store username password ==

RegistrationForm.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="RegistrationForm.aspx.cs"
Inherits="WebApplication.RegistrationForm" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
   <title></title>
</head>
<body>
   <form id="form1" runat="server">
      <div>
         <h2>Registration Form</h2>

         <h5>Name : </h5>
         <asp:TextBox runat="server" ID="txt1" Width="200"/>
         <br /><br />

         <h5>Password</h5>
         <asp:TextBox runat="server" ID="txt2" Width="200" TextMode="Password"/>
         <br /><br />
         <asp:Button runat="server" Text="Submit" ID="btn1" OnClick="btn_Submit" Width="100"/>

         <br /><br />

         <asp:Label ID="lbl1" runat="server" ForeColor="Green" />
      </div>
   </form>
</body>
</html>
```

Database table code **database name is dotnet5thsem**

```
USE dotnet5thsem;


CREATE TABLE Users

(

   Id INT PRIMARY KEY IDENTITY(1,1),

   Name VARCHAR(100),

   Password VARCHAR(100)

);
```

Web.config

```
<!-- the name is use in the .cs file for connection and intitial catalog is database name
 Syntax :
<configuration>
<connectionStrings> <add name = “your connectionStringName” connectionString =
“Data Source = DatabaseServerName; Integrated Security = true;
Initial Catalog = YouDatabaseName;
 uid= YourUserName; Password=yourpassword;”
 providerName= “System.Data.SqlClient”/>
</connectionStrings>
```

```
</configuration>
-->


<configuration>
        <connectionStrings>
                <add
                        name="MyConnection"
                        connectionString="
                Data Source=.\SQLEXPRESS;
                Initial Catalog= dotnet5thsem;
                Integrated Security=True
                "
                        providerName="System.Data.SqlClient"
                />
        </connectionStrings>
</configuration>
```

RegistrationForm.aspx.css

```
using System;
using System.Web.UI;
using System.Configuration;
using System.Data.SqlClient;

namespace WebApplication
{
    public partial class RegistrationForm : Page
    {
        protected void Page_Load(Object sender,EventArgs e) { }

        protected void btn_Submit(Object sender,EventArgs e)
        {
            string connString =
ConfigurationManager.ConnectionStrings["MyConnection"].ConnectionString;
            // MyConnection should same as what i use in the configuration file

            SqlConnection conn = new SqlConnection(connString);

            // for input from form
            string name = txt1.Text;
            string pass = txt2.Text;

            string query = "INSERT INTO Users(Name,Password) VALUES (@Name,@Pass)";

            SqlCommand cmd = new SqlCommand(query, conn);
            cmd.Parameters.AddWithValue("@Name", name);
            cmd.Parameters.AddWithValue("@Pass", pass);

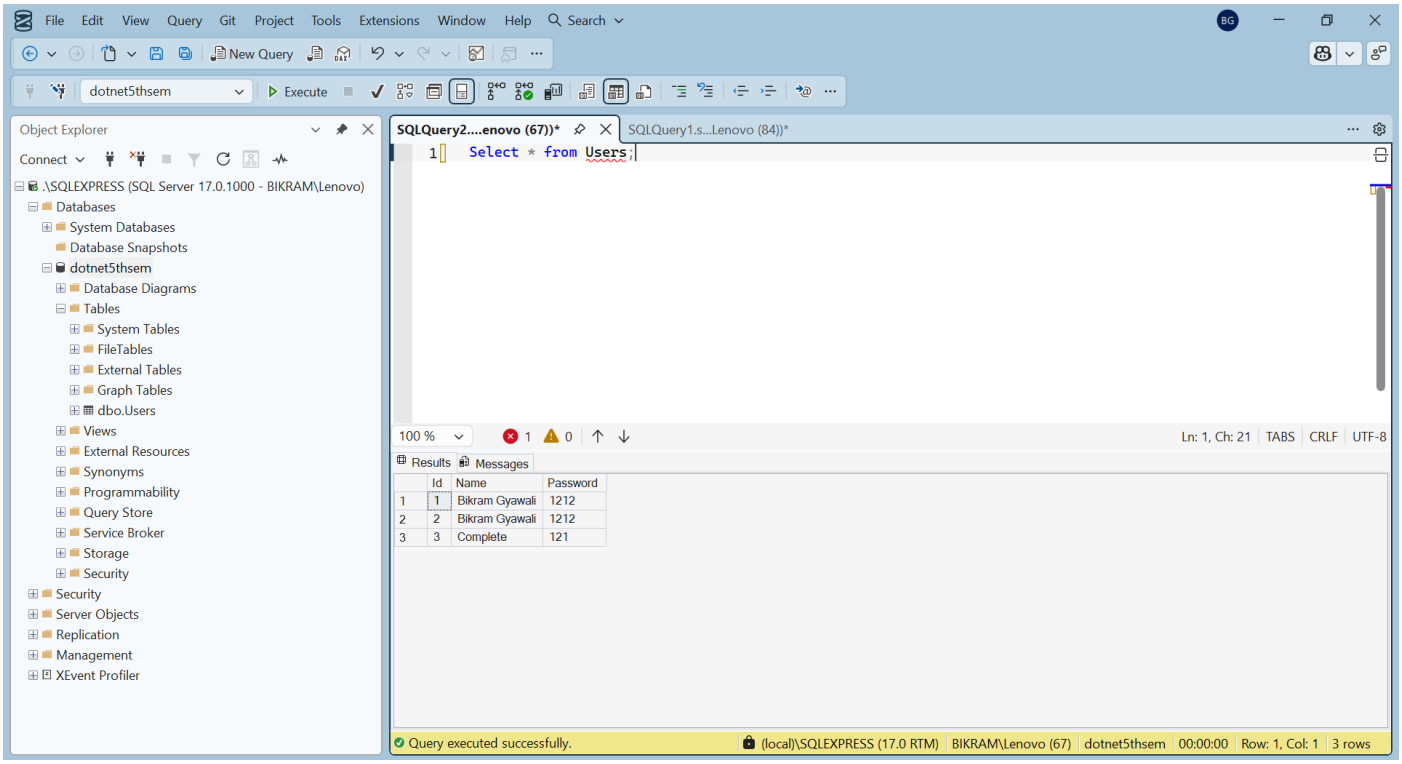
            conn.Open();

            cmd.ExecuteNonQuery();
            conn.Close();
            lbl1.Text = "Record Saved Successfully";

            txt1.Text = "";  //to make the field empty after save
            txt2.Text = "";


        }
    }
}
```


https://localhost:44368/RegistrationForm
Registration Form
Name :
Complete
Password
Submit


File Edit View Query Git Project Tools Extensions Window Help Search
New Query
dotnet5thsem
Execute
Object Explorer
Connect
.\SQLEXPRESS (SQL Server 17.0.1000 - BIKRAM\Lenovo)
Databases
System Databases
Database Snapshots
dotnet5thsem
Database Diagrams
Tables
System Tables
FileTables
External Tables
Graph Tables
dbo.Users
Views
External Resources
Synonyms
Programmability
Query Store
Service Broker
Storage
Security
Security
Server Objects
Replication
Management
XEvent Profiler
SQLQuery2....enovo (67))*
SQLQuery1.s...Lenovo (84))*
1 Select * from Users;
100 %
Ln: 1, Ch: 21
TABS
CRLF
UTF-8
Results
Messages
Id Name Password
1 1 Bikram Gyawali 1212
2 2 Bikram Gyawali 1212
3 3 Complete 121
Query executed successfully.
(local)\SQLEXPRESS (17.0 RTM)
BIKRAM\Lenovo (67)
dotnet5thsem
00:00:00
Row: 1, Col: 1
3 rows